

PEMROGRAMAN BERORIENTASI OBJEK

PRAKTIKUM

NAMA : IDHAM KHOLID NUR AZIZI

NIM : 20230801192

CR002, CSF308

A. LAPORAN PROGRAM FUNGSI BALIK ARRAY

```
package tugas;

import java.util.Arrays;

public class balikArray {
    public static void balikArray(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n / 2; i++) {
            int temp = arr[i];
            arr[i] = arr[n - i - 1];
            arr[n - i - 1] = temp;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        System.out.println("Sebelum dibalik: " + Arrays.toString(arr));
        balikArray(arr);
        System.out.println("Setelah dibalik: " + Arrays.toString(arr));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk membalik urutan elemen di dalam array. Misalnya, jika array berisi {1, 2, 3, 4, 5}, maka setelah proses pembalikan, array tersebut akan menjadi {5, 4, 3, 2, 1}.

2. Penjelasan Fungsi

- `balikArray(int[] arr)`: Fungsi ini menerima sebuah array sebagai parameter dan melakukan pembalikan urutan elemen array secara langsung tanpa membuat array baru.
 - Fungsi memanfaatkan teknik swap (pertukaran), di mana elemen pertama akan bertukar dengan elemen terakhir, elemen kedua bertukar dengan elemen kedua dari belakang, dan seterusnya.
 - Perulangan dilakukan hanya hingga setengah panjang array untuk menghindari elemen kembali ke posisi semula.
- `main(String[] args)`: Merupakan fungsi utama dari program.
 - Sebuah array dengan elemen {1, 2, 3, 4, 5} didefinisikan dan dicetak sebelum serta sesudah dipanggilnya fungsi `balikArray()`.
 - Metode `Arrays.toString()` digunakan untuk mencetak array dalam bentuk string yang mudah dibaca.

3. Output Program

Sebelum dibalik: [1, 2, 3, 4, 5]

Setelah dibalik: [5, 4, 3, 2, 1]

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Algoritma ini menggunakan perulangan dengan kompleksitas $O(n/2)$, yang sebenarnya sama dengan $O(n)$, di mana n adalah panjang array. Hal ini karena program hanya perlu mengunjungi setengah dari elemen array untuk membalik urutannya.
- Kompleksitas Ruang
 - Kompleksitas ruang dari program ini adalah $O(1)$ karena pembalikan dilakukan langsung pada array yang ada tanpa memerlukan array tambahan atau memori ekstra yang signifikan selain variabel sementara temp.

5. Kesimpulan

Program `balikArray` ini berhasil membalik urutan elemen pada array dengan menggunakan algoritma yang efisien. Dengan pendekatan swapping (pertukaran), elemen-elemen array dapat dibalik dalam waktu $O(n)$. Program ini sederhana namun berguna sebagai dasar pemahaman tentang manipulasi array di Java.

B. LAPORAN PROGRAM FUNGSI CEK PRIMA

```
package tugas;

public class cekPrima {
    public static boolean cekPrima(int angka) {
        if (angka <= 1) return false;
        for (int i = 2; i <= Math.sqrt(angka); i++) {
            if (angka % i == 0) return false;
        }
        return true;
    }

    public static void cetakPrima(int batasAtas) {
        for (int i = 2; i <= batasAtas; i++) {
            if (cekPrima(i)) {
                System.out.print(i + " ");
            }
        }
    }

    public static void main(String[] args) {
        int batasAtas = 10;
        System.out.println("Bilangan prima hingga " + batasAtas + ":");
        cetakPrima(batasAtas);
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk mengecek apakah suatu bilangan merupakan bilangan prima, serta mencetak semua bilangan prima hingga batas atas yang ditentukan.

2. Penjelasan Fungsi

- cekPrima(int angka): Fungsi ini menerima sebuah bilangan dan mengembalikan true jika bilangan tersebut merupakan bilangan prima dan false jika bukan.
 - Bilangan prima adalah bilangan yang hanya memiliki dua faktor, yaitu 1 dan dirinya sendiri.
 - Fungsi ini memeriksa apakah bilangan habis dibagi oleh bilangan lain dari 2 hingga akar kuadrat dari bilangan tersebut.
- cetakPrima(int batasAtas): Fungsi ini mencetak semua bilangan prima dari 2 hingga batas atas yang diberikan.
 - Memanfaatkan fungsi cekPrima() untuk memeriksa setiap bilangan dari 2 hingga batas atas.
 - Jika bilangan tersebut adalah prima, maka bilangan dicetak.
- main(String[] args): Fungsi utama program yang menetapkan nilai batas atas, kemudian mencetak daftar bilangan prima hingga batas tersebut menggunakan fungsi cetakPrima().

3. Output Program

Bilangan prima hingga 10:

2 3 5 7

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Fungsi cekPrima() memiliki kompleksitas $O(\sqrt{n})$, di mana n adalah bilangan yang diperiksa. Hal ini karena program hanya perlu memeriksa pembagi hingga akar kuadrat dari bilangan tersebut.
 - Fungsi cetakPrima() melakukan perulangan untuk setiap bilangan dari 2 hingga batas atas b , sehingga kompleksitas waktu total program adalah $O(b\sqrt{b})$, di mana b adalah batas atas.
- Kompleksitas Ruang:
 - Kompleksitas ruang adalah $O(1)$, karena program tidak memerlukan struktur data tambahan yang signifikan selain variabel loop dan variabel sementara.

5. Kesimpulan

Program cekPrima ini efektif dalam memeriksa dan mencetak bilangan prima hingga batas tertentu. Dengan menggunakan optimasi melalui pengecekan hingga akar kuadrat, program berjalan dengan efisiensi yang baik untuk batas atas yang relatif kecil. Program ini memberikan dasar pemahaman yang kuat tentang pengecekan bilangan prima di Java, namun bisa ditingkatkan lebih lanjut untuk menangani batas atas yang lebih besar dengan algoritma yang lebih canggih.

C. LAPORAN PROGRAM FUNGSI CELCIUS TO FAHRENHEIT

```
package tugas;

public class celciusToFahrenheit {
    public static double celciusToFahrenheit(double celcius) {
        return (celcius * 9/5) + 32;
    }

    public static void main(String[] args) {
        double celcius = 10.0;
        System.out.println(celcius + " derajat Celcius sama dengan " +
            celciusToFahrenheit(celcius) + " derajat Fahrenheit.");
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk mengkonversi suhu dari satuan Celcius ke Fahrenheit menggunakan rumus standar.

2. Penjelasan Fungsi

- `celciusToFahrenheit(double celcius)`: Fungsi ini menerima suhu dalam Celcius sebagai input dan mengembalikannya dalam satuan Fahrenheit.
 - `main(String[] args)`: Fungsi utama program yang menetapkan nilai suhu dalam Celcius dan mencetak hasil konversinya ke dalam Fahrenheit dengan memanggil fungsi `celciusToFahrenheit()`.

3. Output Program

10.0 derajat Celcius sama dengan 50.0 derajat Fahrenheit.

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu dari program ini adalah $O(1)$, karena operasi aritmatika sederhana yang dilakukan untuk konversi suhu tidak bergantung pada ukuran input. Konversi selalu dilakukan dalam waktu konstan.
- Kompleksitas Ruang:
 - Kompleksitas ruang dari program juga $O(1)$, karena hanya membutuhkan memori untuk menyimpan beberapa variabel (`celcius` dan hasil perhitungan dalam Fahrenheit).

5. Kesimpulan

Program `celciusToFahrenheit` ini merupakan solusi yang sederhana namun efektif untuk mengonversi suhu dari satuan Celcius ke Fahrenheit. Program ini menggunakan rumus standar dalam konversi suhu dan mampu mengembalikan hasil dengan cepat. Meski cukup terbatas dalam fungsinya, program ini bisa menjadi dasar untuk pengembangan aplikasi konversi suhu yang lebih lengkap.

D. LAPORAN PROGRAM FUNGSI FAKTORIAL

```
package tugas;

public class faktorial {
    public static int faktorial(int n) {
        if (n == 0) {
            return 1;
        } else {
            return n * faktorial(n - 1);
        }
    }

    public static void main(String[] args) {
        int angka = 4;
        System.out.println("Faktorial dari " + angka + " adalah: " +
            faktorial(angka));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung nilai faktorial dari suatu bilangan integer menggunakan pendekatan rekursif.

2. Penjelasan Fungsi

- faktorial(int n): Fungsi ini menerima bilangan integer n dan mengembalikan hasil faktorial dari n. Faktorial didefinisikan sebagai:
 - $n! = n \times (n-1) \times (n-2) \times \dots \times 1$] atau jika $n = 0$, faktorial dari 0 adalah 1, yaitu: $0! = 1$]
- main(String[] args): Fungsi utama yang menetapkan nilai n dan mencetak hasil faktorial dari n dengan memanggil fungsi faktorial().

3. Output Program

Faktorial dari 4 adalah: 24

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu dari program ini adalah $O(n)$, di mana n adalah bilangan yang diinput. Hal ini karena program memanggil fungsi faktorial secara rekursif sebanyak n kali hingga mencapai kondisi dasar.
- Kompleksitas Ruang:
 - Kompleksitas ruang dari program juga $O(n)$, karena setiap panggilan rekursif menyimpan state atau status di tumpukan (stack) hingga mencapai kondisi dasar rekursi.

5. Kesimpulan

Program faktorial ini menggunakan pendekatan rekursif untuk menghitung faktorial suatu bilangan dengan efisien. Dengan adanya kondisi dasar yang menangani kasus $n = 0$, program ini berhasil menyelesaikan perhitungan rekursif dengan benar. Namun, untuk input yang lebih besar atau nilai negatif, program bisa ditingkatkan lebih lanjut dengan validasi input dan alternatif metode iteratif.

E. LAPORAN PROGRAM FUNGSI FIBONACCI

```
package tugas;

public class fibonacci {
    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        int n = 7;
        System.out.println("Fibonacci ke-" + n + " adalah: " + fibonacci(n));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung nilai deret Fibonacci pada indeks ke-n menggunakan pendekatan rekursif.

2. Penjelasan Fungsi

- `fibonacci(int n)`: Fungsi ini menerima bilangan n dan mengembalikan nilai Fibonacci ke- n . Deret Fibonacci didefinisikan sebagai:
 $F(0) = 0$, $F(1) = 1$
- Untuk nilai $n > 1$, Fibonacci ke- n dihitung dengan rumus rekursif
- `main(String[] args)`: Fungsi utama program yang menetapkan nilai n dan mencetak hasil Fibonacci ke- n dengan memanggil fungsi `fibonacci()`.

3. Output Program

Fibonacci ke-7 adalah: 13

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu dari program ini adalah $O(2^n)$, karena program memanggil fungsi `fibonacci` secara rekursif dua kali untuk setiap nilai n (yaitu `fibonacci(n-1)` dan `fibonacci(n-2)`). Ini menghasilkan banyak perhitungan ulang untuk nilai Fibonacci yang sama, membuat program tidak efisien untuk nilai n yang besar.
- Kompleksitas Ruang:
 - Kompleksitas ruang dari program adalah $O(n)$, karena rekursi menggunakan ruang memori yang sesuai dengan kedalaman panggilan rekursif.

5. Kesimpulan

Program `fibonacci` ini menggunakan metode rekursif untuk menghitung nilai Fibonacci ke- n . Meskipun kode ini sederhana dan mudah dimengerti, pendekatan rekursif ini sangat tidak efisien karena mengulang perhitungan yang sama berkali-kali. Untuk perhitungan Fibonacci pada indeks besar, disarankan menggunakan metode optimasi seperti memoization atau iterasi agar lebih efisien.

F. LAPORAN PROGRAM FUNGSI HITUNG KATA

```
package tugas;

public class hitungKata {
    public static int hitungKata(String kalimat) {
        if (kalimat == null || kalimat.isEmpty()) {
            return 0;
        }
        String[] kata = kalimat.split("\\s+");
        return kata.length;
    }

    public static void main(String[] args) {
        String kalimat = "Belajar Java itu menyenangkan dan menarik";
        System.out.println("Jumlah kata: " + hitungKata(kalimat));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung jumlah kata dalam sebuah string atau kalimat yang diinputkan oleh pengguna. Kata-kata dipisahkan oleh spasi.

2. Penjelasan Fungsi

- `hitungKata(String kalimat)`: Fungsi ini menerima sebuah string (kalimat) dan mengembalikan jumlah kata dalam kalimat tersebut. Program ini memanfaatkan metode `split()` untuk memisahkan kata berdasarkan spasi.
- `main(String[] args)`: Fungsi utama yang mendefinisikan kalimat contoh dan mencetak hasil dari fungsi `hitungKata()`.

3. Output Program

Jumlah kata: 6

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu dari program ini adalah $O(n)$, di mana n adalah panjang string kalimat. Hal ini karena program perlu memproses setiap karakter dalam string untuk memisahkan kata-kata berdasarkan spasi.
- Kompleksitas Ruang:
 - Kompleksitas ruang program juga $O(n)$, karena program perlu menyimpan array `kata[]` yang berisi semua kata yang dihasilkan dari pemisahan string.

5. Kesimpulan

Program `hitungKata` ini dirancang untuk menghitung jumlah kata dalam kalimat dengan cara sederhana dan efisien. Penggunaan metode `split("\\s+")` memastikan bahwa spasi ganda atau lebih tidak mengganggu hasil perhitungan. Untuk aplikasi yang lebih kompleks, seperti pemrosesan teks dengan tanda baca, diperlukan pengembangan lebih lanjut, seperti menghilangkan tanda baca sebelum menghitung kata-kata dalam kalimat.

G. LAPORAN PROGRAM FUNGSI HITUNG RATA-RATA

```
package tugas;

public class hitungRataRata {
    public static double hitungRataRata(int[] angka) {
        int total = 0;
        for (int n : angka) {
            total += n;
        }
        return (double) total / angka.length;
    }

    public static void main(String[] args) {
        int[] nilai = {80, 90, 70, 85, 95};
        System.out.println("Rata-rata nilai adalah: " +
            hitungRataRata(nilai));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung rata-rata dari beberapa nilai (integer) yang diberikan dalam sebuah array.

2. Penjelasan Fungsi

- `hitungRataRata(int[] angka)`: Fungsi ini menerima array yang berisi sekumpulan angka (tipe integer) dan mengembalikan hasil rata-rata dari nilai-nilai tersebut dalam bentuk double.
- `main(String[] args)`: Fungsi utama yang mendefinisikan array contoh nilai-nilai dan mencetak hasil dari fungsi `hitungRataRata()`.

3. Output Program

Rata-rata nilai adalah: 84.0

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu dari program ini adalah $O(n)$, di mana n adalah panjang array angka. Hal ini karena program perlu melakukan iterasi melalui setiap elemen dalam array satu kali untuk menghitung total.
- Kompleksitas Ruang:
 - Kompleksitas ruang adalah $O(1)$, karena penggunaan ruang tambahan hanya untuk variabel total dan tidak ada penggunaan memori tambahan yang bergantung pada ukuran input (selain array input itu sendiri).

5. Kesimpulan

Program `hitungRataRata` ini dirancang untuk menghitung rata-rata dari sekumpulan angka yang disediakan dalam sebuah array. Fungsi ini berjalan dengan efisien, memiliki kompleksitas waktu $O(n)$, dan cukup mudah untuk dipahami serta digunakan.

H. LAPORAN PROGRAM FUNGSI HITUNG UMUR

```
package tugas;

import java.util.Calendar;

public class hitungUmur {
    public static int hitungUmur(int tahunLahir) {
        int tahunSekarang = Calendar.getInstance().get(Calendar.YEAR);
        return tahunSekarang - tahunLahir;
    }

    public static void main(String[] args) {
        int tahunLahir = 2004;
        System.out.println("Umur Anda adalah: " + hitungUmur(tahunLahir) + " "
            tahun.");
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung usia seseorang dengan menggunakan tahun lahir dan tahun saat ini.

2. Penjelasan Fungsi

- `hitungUmur(int tahunLahir)`: Fungsi ini menghitung selisih antara tahun saat ini dan tahun lahir yang diberikan untuk mendapatkan umur seseorang.
- `main(String[] args)`: Fungsi utama yang memberikan contoh tahun lahir dan menampilkan hasil perhitungan umur.

3. Output Program

Umur Anda adalah: 20 tahun.

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu program ini adalah $O(1)$ karena fungsi `hitungUmur` hanya melakukan operasi perhitungan sederhana yang memakan waktu konstan, yaitu mendapatkan tahun saat ini dan menghitung selisihnya dengan tahun lahir.
- Kompleksitas Ruang:
 - Kompleksitas ruang juga $O(1)$ karena program hanya menggunakan variabel sederhana untuk menyimpan nilai tahun dan hasil perhitungan umur.

5. Kesimpulan

Program `hitungUmur` ini dirancang untuk menghitung umur berdasarkan tahun kelahiran. Kelebihan utamanya adalah kesederhanaan dan efisiensinya dalam menghitung umur dengan waktu dan ruang yang konstan. Namun, perhitungan umur hanya dalam satuan tahun penuh dan tidak memperhitungkan bulan dan hari. Untuk perhitungan umur yang lebih presisi, API modern seperti `LocalDate` dari Java dapat digunakan.

I. LAPORAN PROGRAM FUNGSI MENGHITUNG PANGKAT

```
package tugas;

public class menghitungPangkat {
    public static int pangkat(int base, int eksponen) {
        int hasil = 1;
        for (int i = 0; i < eksponen; i++) {
            hasil *= base;
        }
        return hasil;
    }

    public static void main(String[] args) {
        int base = 5, eksponen = 3;
        System.out.println(base + " pangkat " + eksponen + " adalah: " +
pangkat(base, eksponen));
    }
}
```

1. Tujuan Program

Program ini bertujuan untuk menghitung hasil dari suatu bilangan yang dipangkatkan dengan eksponen tertentu menggunakan perulangan.

2. Penjelasan Fungsi

- `pangkat(int base, int eksponen)`: Fungsi ini menghitung pangkat dari base dengan eksponen menggunakan loop for.
- `main(String[] args)`: Fungsi utama yang memberikan contoh bilangan basis (base) dan eksponen yang akan dipangkatkan serta menampilkan hasilnya.

3. Output Program

5 pangkat 3 adalah: 125

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Kompleksitas waktu program ini adalah $O(n)$, di mana n adalah nilai dari eksponen. Hal ini karena perulangan terjadi sebanyak eksponen kali untuk menghitung hasil pangkat.
- Kompleksitas Ruang:
 - Kompleksitas ruang adalah $O(1)$, karena program hanya menggunakan beberapa variabel untuk menyimpan nilai sementara (hasil, base, dan eksponen) tanpa alokasi memori tambahan yang bergantung pada ukuran input.

5. Kesimpulan

Program `menghitungPangkat` ini merupakan solusi sederhana dan efisien untuk menghitung pangkat suatu bilangan dengan menggunakan perulangan. Kelebihan utama dari program ini adalah kemudahannya dalam implementasi dan efisiensinya dalam hal kompleksitas ruang. Namun, peningkatan dapat dilakukan dengan menambahkan dukungan untuk eksponen negatif dan menggunakan metode rekursi sebagai alternatif pendekatan.

J. LAPORAN PROGRAM FUNGSI PERSEGI PANJANG

```
package tugas;

public class persegiPanjang {
    public static int hitungKeliling(int panjang, int Lebar) {
        return 2 * (panjang + Lebar);
    }

    public static int hitungLuas(int panjang, int Lebar) {
        return panjang * Lebar;
    }

    public static void main(String[] args) {
        int panjang = 10, lebar = 5;
        System.out.println("Keliling persegi panjang: " +
hitungKeliling(panjang, lebar));
        System.out.println("Luas persegi panjang: " + hitungLuas(panjang,
lebar));
    }
}
```

1. Tujuan Program

Program ini dirancang untuk menghitung keliling dan luas dari sebuah persegi panjang berdasarkan nilai panjang dan lebar yang diberikan.

2. Penjelasan Fungsi

- `hitungKeliling(int panjang, int lebar)`: Fungsi ini digunakan untuk menghitung keliling persegi panjang menggunakan rumus $Keliling = 2 \times (Panjang + Lebar)$.
- `hitungLuas(int panjang, int lebar)`: Fungsi ini digunakan untuk menghitung luas persegi panjang dengan rumus $Luas = Panjang \times Lebar$.
- `main(String[] args)`: Fungsi utama yang menjalankan perhitungan keliling dan luas berdasarkan input nilai panjang dan lebar yang ditetapkan dalam kode.

3. Output Program

Keliling persegi panjang: 30

Luas persegi panjang: 50

4. Analisis Kinerja Program

- Kompleksitas Waktu:
 - Fungsi `hitungKeliling` dan `hitungLuas` masing-masing memiliki kompleksitas waktu $O(1)$, karena hanya melibatkan operasi aritmatika sederhana.
- Kompleksitas Ruang:
 - Program ini juga memiliki kompleksitas ruang $O(1)$, karena tidak menggunakan struktur data tambahan yang besar selain dari variabel panjang, lebar, dan hasil kalkulasi.

5. Kesimpulan

Program `persegiPanjang` ini merupakan solusi yang sederhana dan efisien untuk menghitung keliling dan luas persegi panjang. Kode ini cocok untuk kasus-kasus dasar yang melibatkan bilangan bulat, tetapi dapat ditingkatkan lebih lanjut dengan menambahkan validasi input dan dukungan untuk bilangan desimal.